

**Bilkent University, CS 473/573, Session 3, 26pts, 35min**

**Q3:** Describe in pseudocode a **worst case linear time** algorithm that, given  $n$  distinct numbers stored in an array  $A[1..n]$ , and a positive integer  $k \leq n$ , finds and returns the  $k$  numbers in  $A$  that are **closest to the  $k^{\text{th}}$  smallest number** in  $A$ . For the sake of simplicity, you may assume that there exists no two numbers in  $A$  with the same closeness to the  $k^{\text{th}}$  smallest number. Your answer should be structured as follows:

- First the idea,
- Then the algorithm,
- And the run time analysis.

**Substitution Method:**

- Guess the asymptotic complexity.
- Prove your guess using induction
  - Assume inductive hypothesis holds for  $k < n$
  - Try to prove the general case for  $n$ 

**Note:** You MUST prove the EXACT inequality, you CANNOT ignore lower order terms. If the proof fails, you could strengthen the inductive hypothesis and try again.

**Master Method:**

$T(n) = aT\left(\frac{n}{b}\right) + f(n)$ ,  $a \geq 1$ ,  $b > 1$ , and  $f$  is asymptotically positive, then:

- $f(n) = O(n^{\log_b a - \epsilon})$ ,  $\epsilon > 0 \Rightarrow T(n) = \Theta(n^{\log_b a})$
- $f(n) = \Theta(n^{\log_b a} \log^k n)$ ,  $k \geq 0 \Rightarrow T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
- $f(n) = \Omega(n^{\log_b a + \epsilon})$ ,  $\epsilon > 0$ , and  $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$ , for some  $c < 1 \Rightarrow T(n) = \Theta(f(n))$

You may use the following algorithms and the like covered in the lectures/book as indicated below:

- $INSERTION-SORT(A, n)$ : Insertion sort on  $A[1..n]$
- $MERGE-SORT(A, p, r)$ : Merge sort on  $A[p..r]$
- $BINARY-SEARCH(A, p, r, key)$ : Binary search for  $key$  on  $A[p..r]$
- $H-PARTITION(A, p, r)$ : Hoare's partitioning on  $A[p..r]$ , returns the last position of the region  $\leq pivot$
- $L-PARTITION(A, p, r)$ : Lomuto's partitioning on  $A[p..r]$ , returns the last position of the region  $\leq pivot$
- $QUICKSORT(A, p, r)$ : Quicksort on  $A[p..r]$
- $R-QUICKSORT(A, p, r)$ : Randomized quicksort on  $A[p..r]$
- $R-H-PARTITION(A, p, r)$ : Randomized Hoare's partitioning on  $A[p..r]$  with a randomly selected pivot, returns the last position of the region  $\leq pivot$
- $R-L-PARTITION(A, p, r)$ : Randomized Lomuto's partitioning on  $A[p..r]$  with a randomly selected pivot, returns the last position of the region  $\leq pivot$
- $R-SELECT(A, p, r, i)$ : Randomized select algorithm on array  $A[p..r]$ , returns the value of the  $i$ th smallest element
- $MM-SELECT(S, n, i)$ : Median of medians algorithm on set  $S$  containing  $n$  elements, returns the value of the  $i$ th smallest element
- $HEAP-EXTRACT-MAX(A, n)$ : Extract max on the heap of  $n$  elements stored in  $A[1..n]$
- $MAX-HEAPIFY(A, i, n)$ : Max heapify on the  $i$ th element of the heap stored in  $A[1..n]$
- $BUILD-MAX-HEAP(A, n)$ : Builds heap in  $A[1..n]$  using the  $n$  elements given in  $A[1..n]$
- $HEAPSORT(A, n)$ : Heapsort on  $A[1..n]$
- $MAX-HEAP-INSERT(A, key, n)$ : Inserts  $key$  into the heap stored in  $A[1..n]$
- $HEAP-INCREASE-KEY(A, i, key)$ : Increases the value of the  $i$ th element of the heap to  $key$
- $COUNTSORT(A, n, k)$ : Applies counting sort on  $A[1..n]$  of integers in range  $[1..k]$